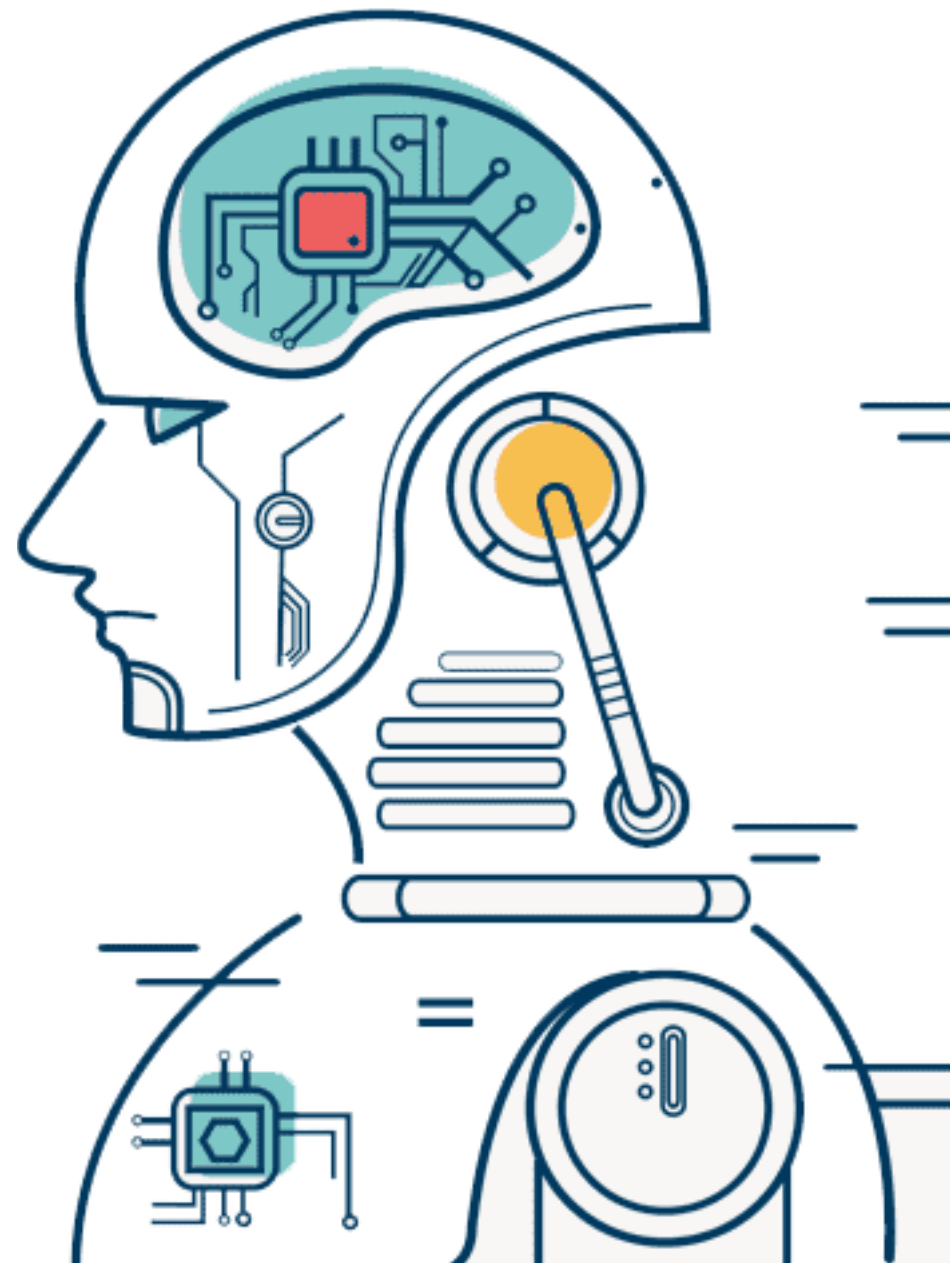


Machine Learning

Chapter04 앙상블 모델, 그리드서치

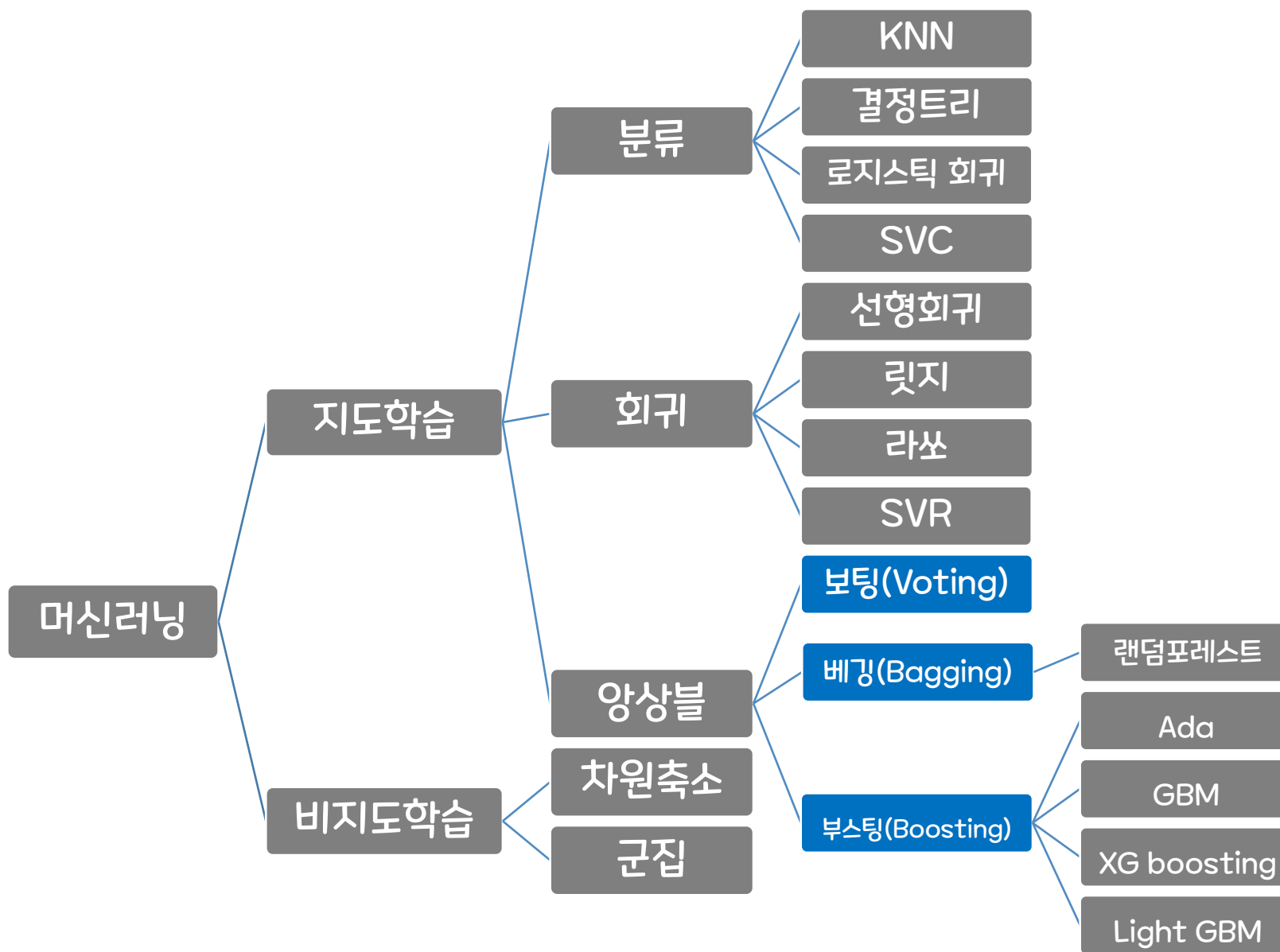
양재원 연구원



- 앙상블(Ensemble) 개념을 이해할 수 있다.
- 앙상블(Ensemble) 모델을 활용하여 학습하는 방법에 대해 이해하고 사용할 수 있다.



머신러닝 모델 개략도





앙상블(Ensemble) 수업 흐름도



개요

- 개념
- 사용이유

앙상블 종류

- 보팅(Voting)
- 배깅(Bagging)
- 부스팅(Boosting)

Bagging모델

- 랜덤 포레스트
(Random forest)

Boosting모델

- Ada boosting
- Gradient boosting(GBM)
- XG boosting
- LightGBM

코드실습

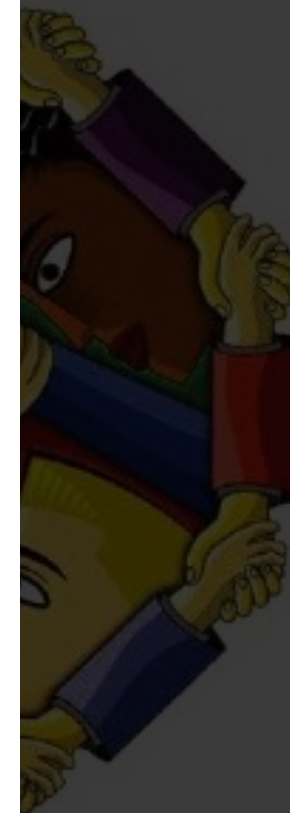


앙상블 (Ensemble)

집단지성



WIKIPEDIA
The Free Encyclopedia





앙상블 (Ensemble)

여러 개의
정확한



결합하여
는 기법



앙상블을 사용하는 이유

1. 단일 모델에 비해 **높은 성능**과 **신뢰성**을 얻을 수 있음
2. 데이터의 양이 적은 것에 대비 **충분한 학습 효과**를 거둘 수 있음



앙상블 (Ensemble)

1. 보팅(Voting)

여러 개의 **다른 종류의** 모델이 예측한 결과를 투표 혹은 평균을 통해 최종 선정

2. 베깅(Bagging)

여러 개의 **같은 종류의** 모델이 예측한 결과를 투표 혹은 평균을 통해 최종 선정

3. 부스팅(Boosting)

여러 개의 **같은 종류의** 모델이 **순차적**으로 학습-예측하며 **오류를 개선**하는 방식



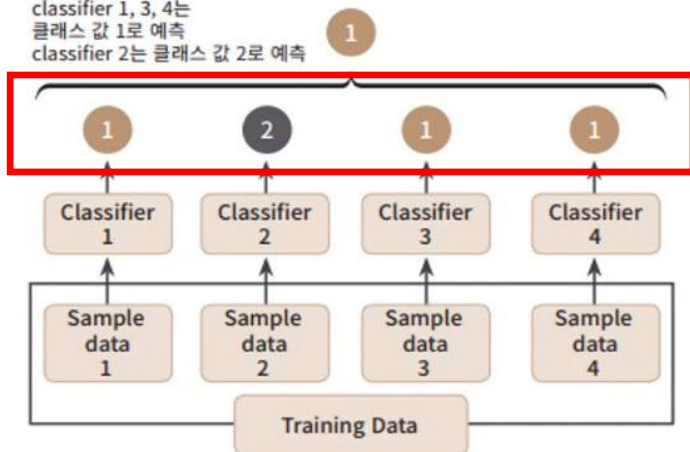
보팅 (Voting)

여러 개의 **다른** 모델이 예측한 결과를 투표 혹은 평균을 통해 최종 예측결과를 선정

1. 하드 보팅(Hard voting) : **다수결**
2. 소프트 보팅(Soft voting) : 각 **확률**의 평균

Hard Voting은 다수의 classifier 간 다수결로 최종 class 결정

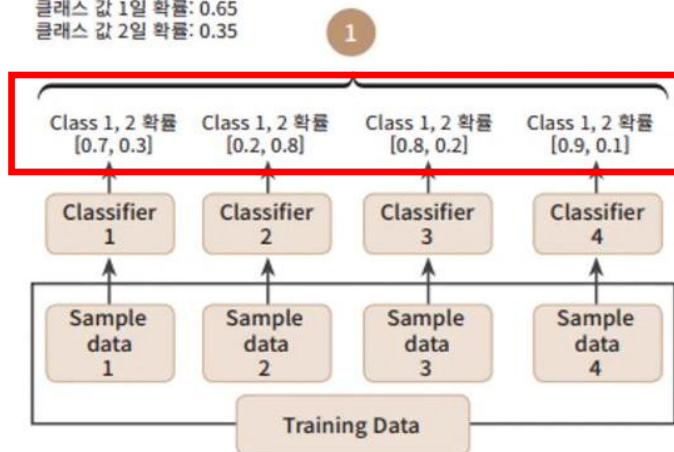
클래스 값 1로 예측
classifier 1, 3, 4는
클래스 값 1로 예측
classifier 2는 클래스 값 2로 예측



<하드 보팅>

Soft Voting은 다수의 classifier 들의 class 확률을 평균하여 결정

클래스 값 1로 예측
클래스 값 1일 확률: 0.65
클래스 값 2일 확률: 0.35



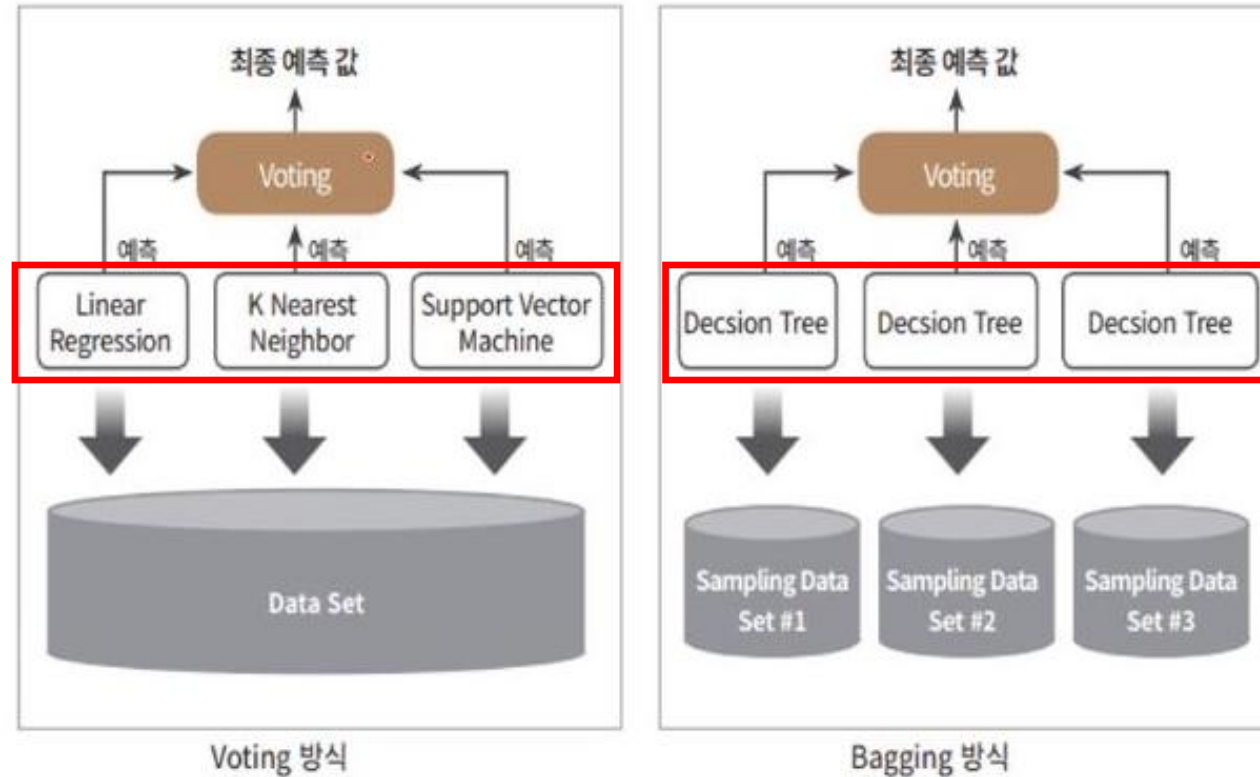
<소프트 보팅>

- 일반적으로 하드 보팅보다는 소프트 보팅이 예측 성능이 상대적으로 우수하여 주로 사용됨.
- 사이킷런은 **VotingClassifier** 클래스를 통해 보팅(Voting)을 지원

회귀 모델은 각 모델들이 예측한 수치값의 평균을 통해서 최종 예측



보팅 (Voting), 베깅 (Bagging)



공통점 : 여러 개의 모델이 투표 혹은 평균을 통해 최종 예측결과를 선정

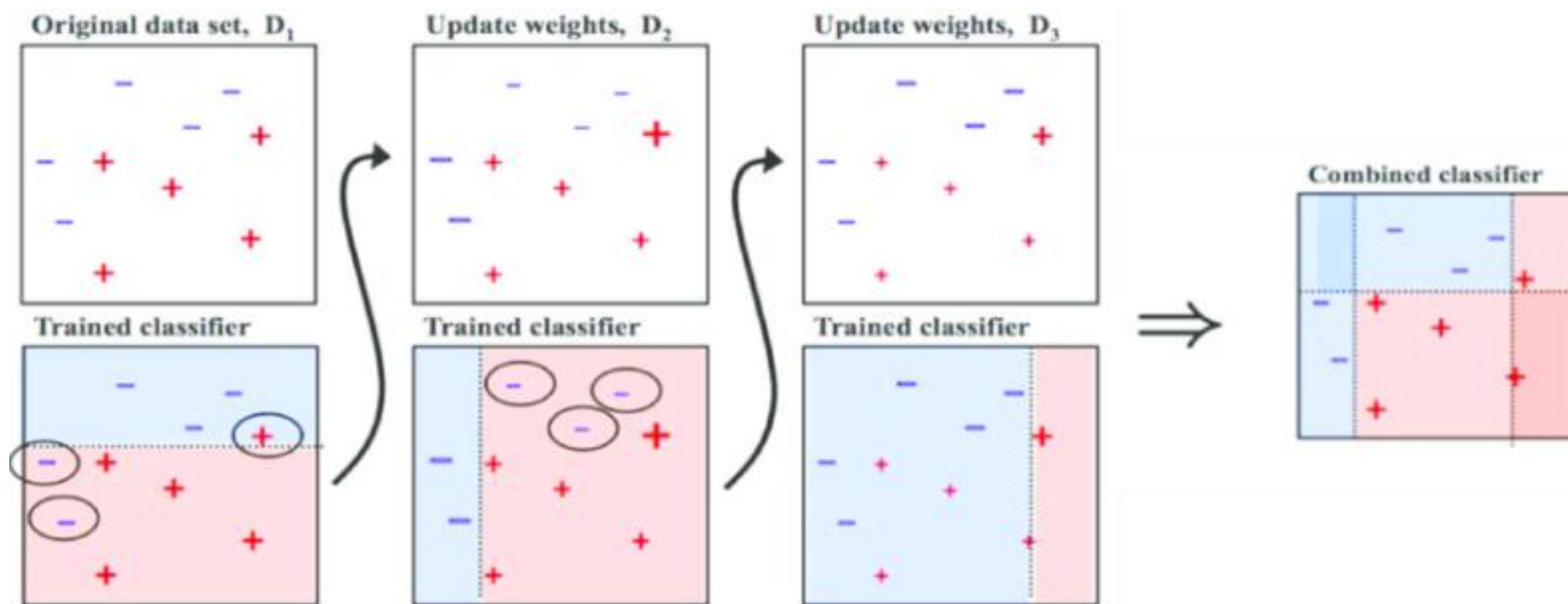
1. Voting : 서로 다른 모델을 결합

2. Bagging : 같은 종류의 모델을 결합(데이터 샘플링을 다르게, 중첩 허용)



부스팅 (Boosting)

여러 개의 모델이 순차적으로 학습-예측하며 잘못 예측한 데이터에 **가중치**를 부여해 **오류를 개선**해 나가면서 학습하는 방식(결정 트리 모형을 베이스로 사용)



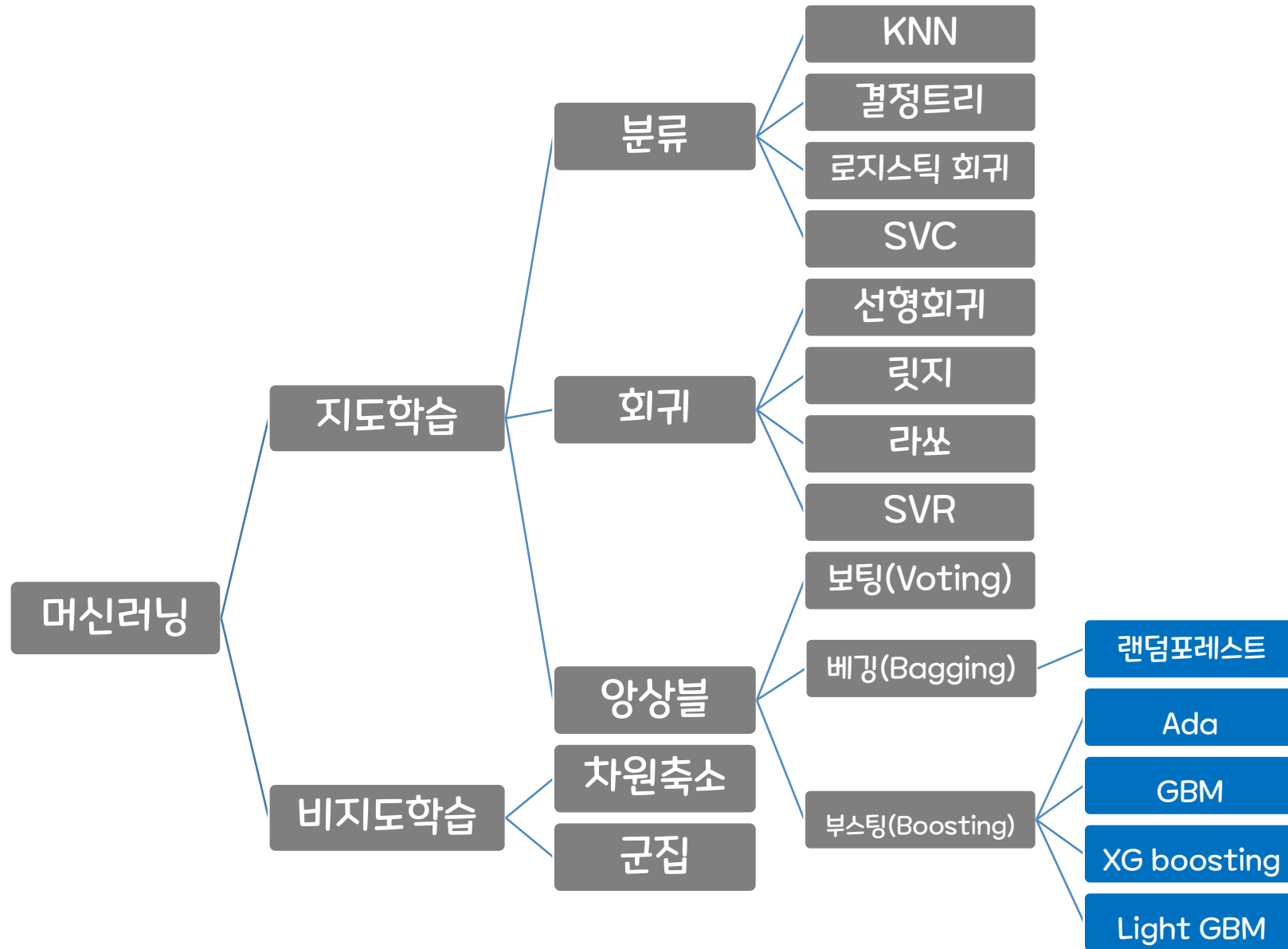


배깅 (Bagging), 부스팅(Boosting)

구 분	배깅(Bagging)	부스팅(Boosting)
특징	• 같은 종류의 모델이 투표를 통해 최종 예측 결과를 도출 (데이터 샘플을 다르게 가져감)	• 순차 학습+예측 (이전 모델의 오류를 고려)
목적	• 일반적으로 좋은 모델을 만들기 위해 • 과대적합 방지(편향된 학습을 방지)	• 맞추기 어려운 문제를 풀기 위해 • 과소적합 방지(부족한 학습을 방지)
적합한 상황	• 데이터 값들의 편차가 클 경우	• 학습 정확도가 낮거나 오차가 클 경우
대표 모델	• Random Forest	• Ada Boosting, Gradient Boosting, XG Boosting, Light GBM
데이터 선택	• 무작위 선택	• 무작위 선택 (오류 데이터에 가중치 적용)



머신러닝 모델 개략도



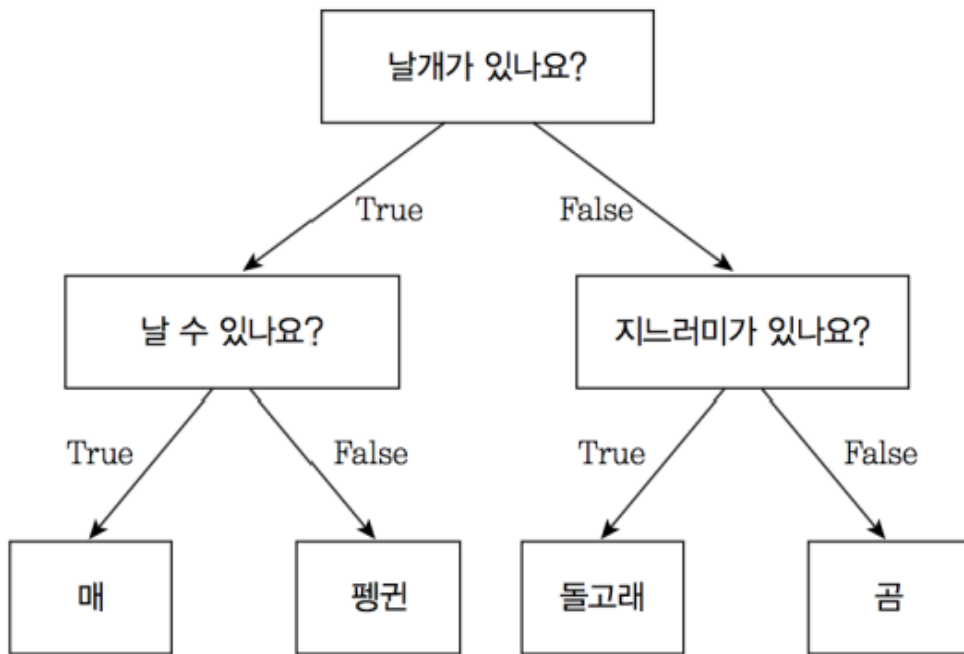


랜덤 포레스트 (Random Forest)

여러 개의 결정 트리 모델로 예측한 값을 투
표를 통해서 최종 선택하는 베깅의 대표적
모델



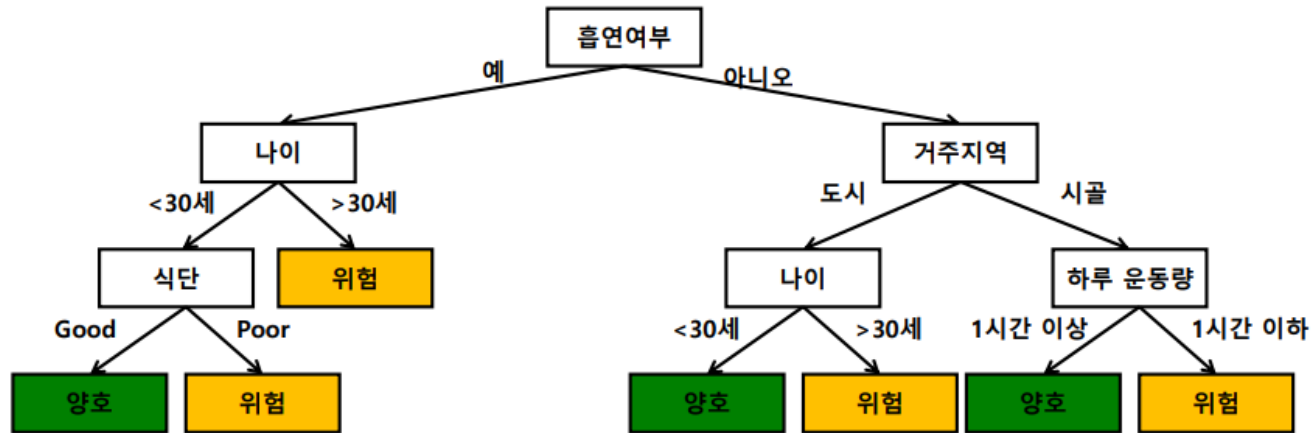
결정 트리(Decision Tree)



- 직관적이어서 결과를 쉽게 이해할 수 있음
- 트리가 깊어질수록 과대적합이 되기 쉬움



랜덤 포레스트(Random Forest)

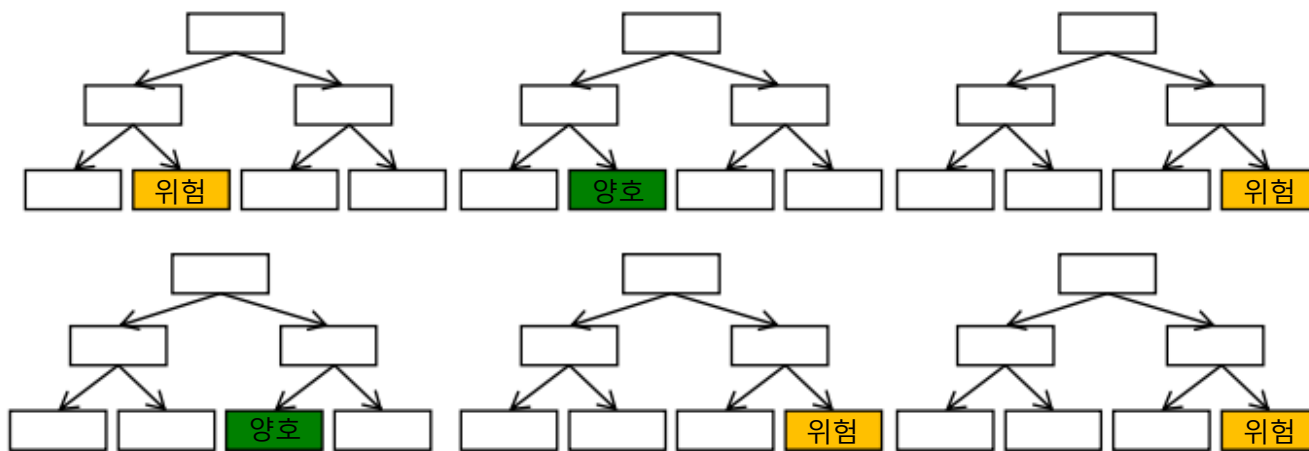


건강 위험도를 예측하기 위한 결정 트리



랜덤 포레스트(Random Forest)

- 다수의 의사결정트리의 의견이 통합되지 않는다면 → 투표에 의한 **다수결의 원칙**을 따름 → **앙상블 방법** (Ensemble Methods)
- 장점 : 실제값에 대한 추정값 오차 평균화, 분산 감소, 과적합 감소



위험으로 판정 (양호 2, 위험 4)



랜덤 포레스트(Random Forest)

주요 매개변수(Hyperparameter)

scikit-learn의 경우

```
RandomForestClassifier(n_estimators,  
max_features, random_state)
```

- 트리의 개수 : `n_estimators`
- 선택할 특징의 최대 수 : `max_features`
(1로 하면 특성을 고려하지 않으며 큰 값이면 DT와 비슷해짐)
- 선택할 데이터의 시드 : `random_state`



랜덤 포레스트(Random Forest)

결정 트리 매개변수(Hyperparameter)

- 트리의 최대 깊이 : `max_depth`
- 말단 노드 최대 개수 : `max_leaf_nodes`
- 말단 노드가 되기 위한 최소 샘플 수 : `min_samples_leaf`



랜덤 포레스트(Random Forest)

특징

- 결정 트리 모델의 과대적합을 통계적 방법으로 해소
- 결정 트리 모델처럼 쉽고 직관적임
- 부스팅 방식에 비해 빠른 수행 속도
- 모델 튜닝을 위한 시간이 많이 필요(하이퍼 파라미터의 종류가 많음)
- 큰 데이터 세트에도 잘 동작하지만 트리 개수가 많아질수록 시간이 오래 걸림

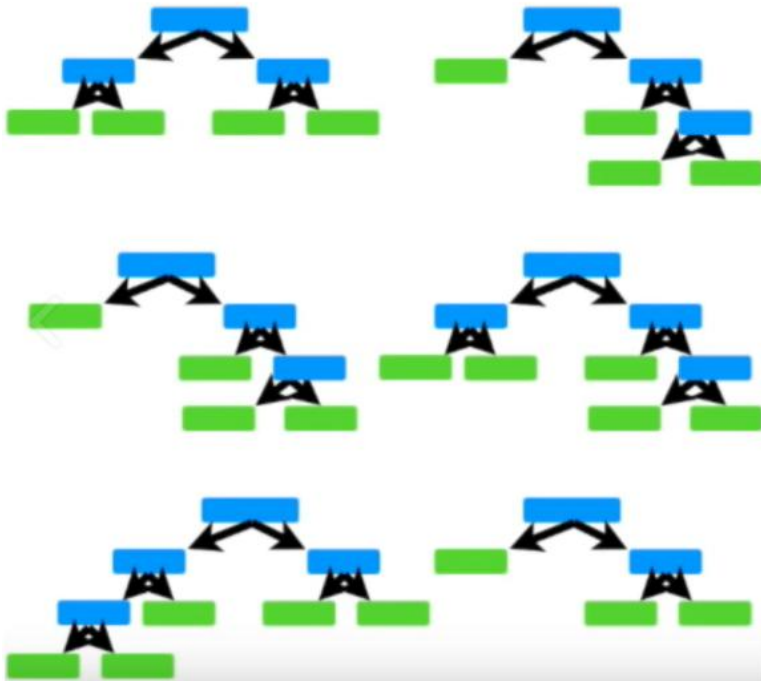


에이다 부스팅 (Ada Boosting)

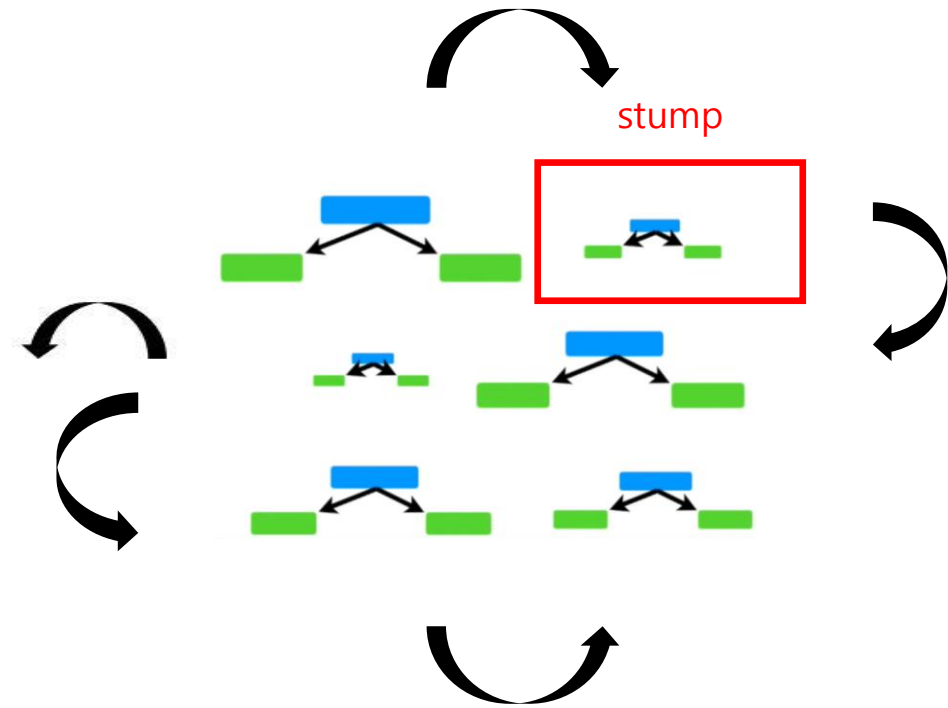
AdaBoost (Adaptive Boosting)

- RF처럼 의사결정 트리 기반의 모델 → 각각의 트리들이 독립적으로 존재하지 않음

약한 학습기(Weak learner)



Random Forest



Ada Boosting

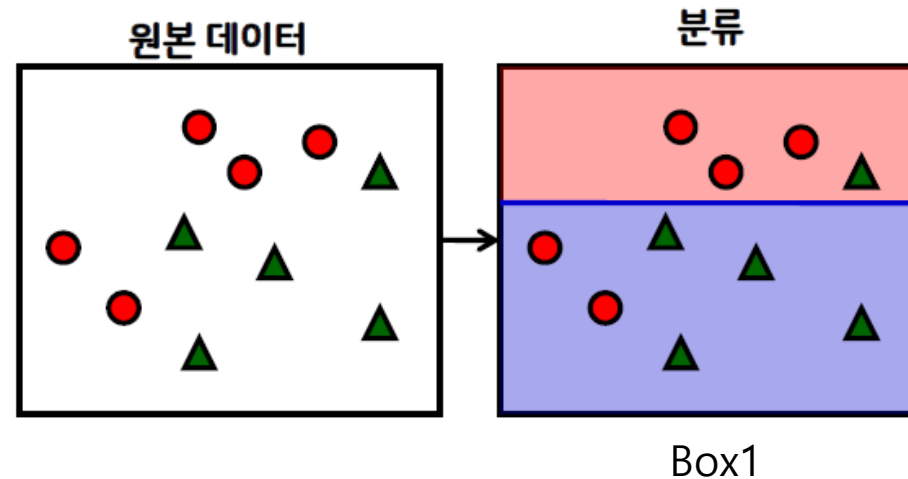


에이다 부스팅 (Ada Boosting)

AdaBoost (Adaptive Boosting)

- 동작 순서

- (1) 첫 번째 의사결정 트리를 생성 → 위쪽 빨간 원이 3개 있는 곳을 대충 분류 시킴
→ 2개의 빨간 원과 1개의 녹색 세모가 잘못 구분됨





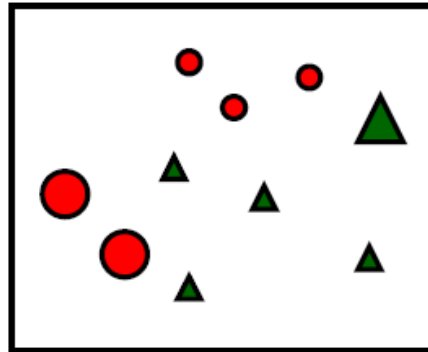
에이다 부스팅 (Ada Boosting)

AdaBoost (Adaptive Boosting)

- 동작 순서

- (2) 잘못된 2개의 빨간 원과 1개의 녹색 세모에 높은 가중치를 부여하고 맞은 것에는 빨간 원 3개와 녹색 세모 4개는 낮은 가중치 부여

오차가 있는 데이터에
가중치 부여



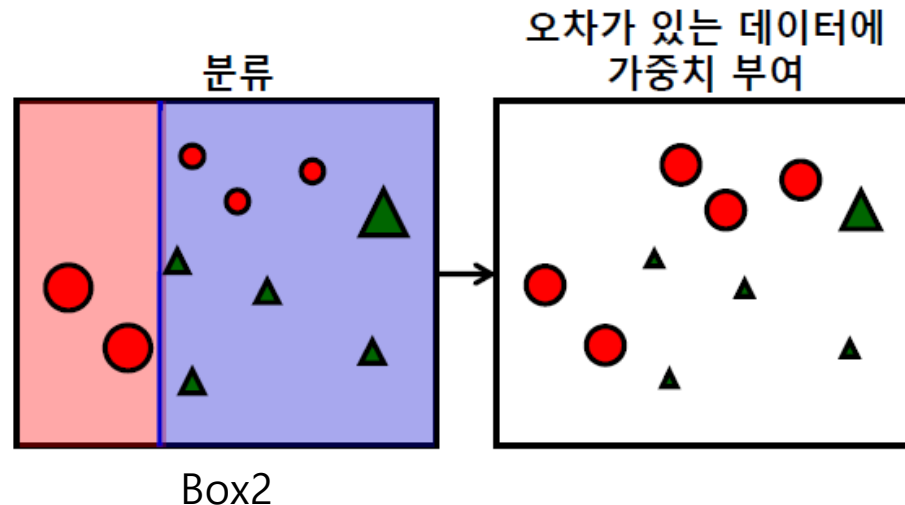


에이다 부스팅 (Ada Boosting)

AdaBoost (Adaptive Boosting)

- 동작 순서

(3) 가중치를 부여한 상태에서 다시 분류 시킴 → 잘못된 3개의 빨간 원에 높은 가중치를 부여하고 맞은 5개의 녹색 세모는 낮은 가중치를 부여



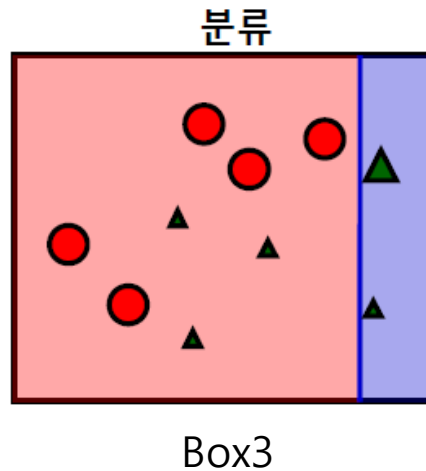


에이다 부스팅 (Ada Boosting)

AdaBoost (Adaptive Boosting)

- 동작 순서

(4) 가중치를 부여한 상태에서 다시 분류 시킴





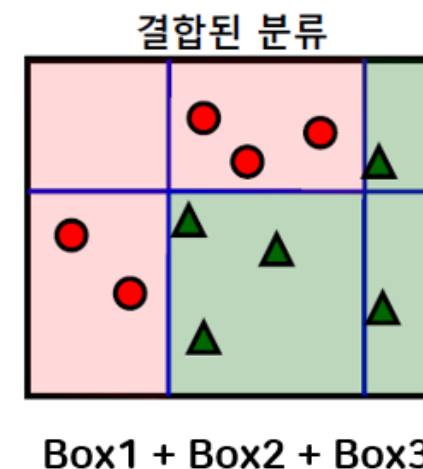
에이다 부스팅 (Ada Boosting)

AdaBoost (Adaptive Boosting)

- 동작 순서

(5) 진행한 분류들을 결합한다.

- 에이다 부스팅은 학습과 예측을 진행할수록 데이터들의 가중치가 달라짐
 - 잘못 분류된 데이터는 가중치 $\uparrow$, 잘 분류된 데이터는 가중치 $\downarrow$
 - 각 트리(stump)들의 가중치 또한 다르게 설정됨
 - 예측률이 높은 트리는 가중치 $\uparrow$, 낮은 트리는 가중치 $\downarrow$
- 예시) $0.4 \cdot \text{Box1} + 0.5 \cdot \text{Box2} + 0.7 \cdot \text{Box3}$





에이다 부스팅 (Ada Boosting)

주요 매개변수(Hyperparameter)

scikit-learn의 경우

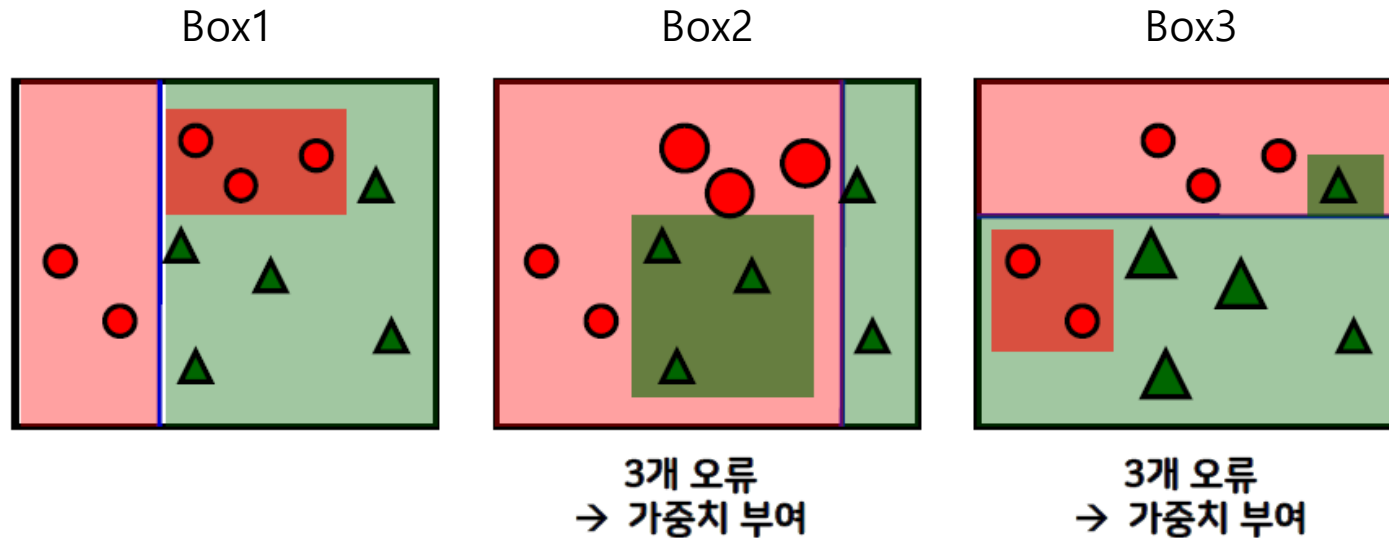
```
AdaBoostClassifier(n_estimators,  
random_state)
```

- 트리의 개수 : `n_estimators`
- 선택할 데이터의 시드 : `random_state`



그레디언트 부스팅 (Gradient Boosting Machine)

- AdaBoost와 기본 개념이 동일하고 가중치를 계산하는 방식에서 **경사하강법**을 이용하여 최적의 가중치(파라미터)를 찾아냄





주요 매개변수(Hyperparameter)

scikit-learn의 경우

```
GradientBoostingClassifier(n_estimators,  
learning_rate, max_depth, random_state)
```

- 트리의 개수 : `n_estimators`
- 학습률 : `learning_rate` (높을수록 오차를 많이 보정)
- 트리의 깊이 : `max_depth`
- 선택할 데이터의 시드 : `random_state`



그레디언트 부스팅 (Gradient Boosting Machine)

장단점

- 학습속도가 느림(부스팅의 일반적인 단점)
- 데이터 특성의 스케일을 조정할 필요가 없음(트리 기반 모델의 특성)



XG 부스팅 (eXtreme Gradient Boosting)

특징

- GBM의 단점 : 느림, 과대적합 문제
- GBM보다 빠름 : Early Stopping 제공
- 과대적합 방지를 위한 규제 포함
- 병렬로 빠른 학습이 가능



XG 부스팅 (eXtreme Gradient Boosting)

주요 매개변수(Hyperparameter)

scikit-learn의 경우

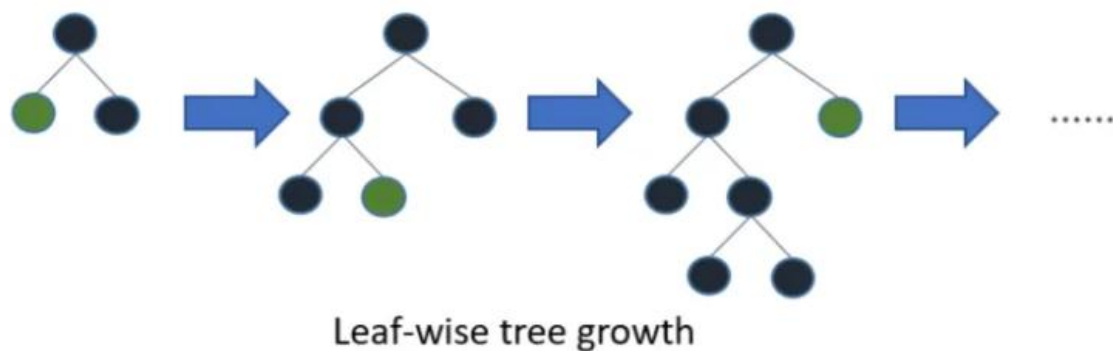
```
XGBClassifier(n_estimators, learning_rate,  
max_depth, random_state)
```

- 트리의 개수 : `n_estimators`
- 학습률 : `learning_rate` (높을수록 오차를 많이 보정)
- 트리의 깊이 : `max_depth`
- 선택할 데이터의 시드 : `random_state`

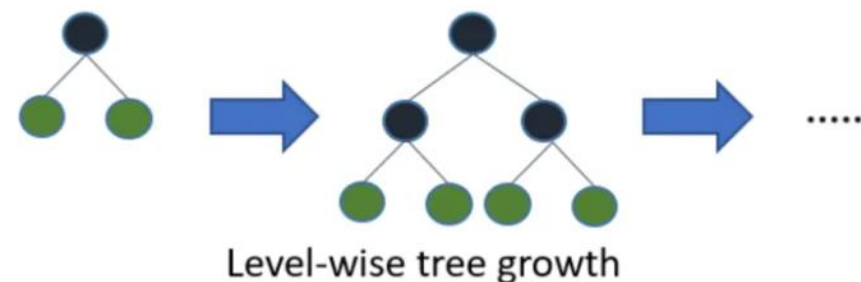


XG Boosting에 비해 **가벼워**(Low memory) **속도가 빠른** 모델

- Leaf-wise(수직방향, 비대칭)로 트리를 성장시킴(속도 ↑)
 - Level-wise(수평방향, 깊이 ↓, 대칭)보다 오류가 더 적음(안정성 ↑)



Light GBM



Random Forest, XG Boosting



장단점

- 대량(1만개 이상)의 데이터를 병렬로 빠르게 학습가능(Low Memory, GPU활용 가능)
 - XG Boosting 대비 2~10배의 속도(동일 파라미터 설정 시)
 - 소량의 데이터에서는 제대로 동작하지 않음(과대적합 위험)
- 예측 속도가 빠름 (Leaf-wise 트리의 장점)
 - 그러나 Level-wise에 비해 과적합에 민감



주요 매개변수(Hyperparameter)

- 100개 이상

- max_depth : 트리의 최대 깊이
- early_stopping_rounds : validation 데이터 중 하나의 지표가 정해진 반복 수 만큼 향상되지 않았다면 학습을 중단
- lambda : lambda 값은 regularization 정규화를 합니다. 일반적인 값의 범위는 0 에서 1 사이
- Min_data_in_leaf : Leaf노드가 가지고 있는 최소한의 레코드 수(디폴트 값 : 20, 과적합을 해결할 때 사용되는 파라미터)
- feature_fraction : 0.8 의 의미는 Light GBM이 Tree를 만들 때 매번 각각의 반복 학습 시 파라미터 중에서 80%를 랜덤하게 선택하는 것을 의미
- bagging_fraction : 매번 iteration을 돌 때 사용되는 데이터의 일부를 선택하는데 트레이닝 속도를 높이고 과적합을 방지할 때 주로 사용
- num_boost_round : boosting 반복 학습 수로 일반적으로 100 이상
- min_gain_to_split : 이 파라미터는 분기하기 위해 필요한 최소한의 gain을 의미, Tree에서 유용한 분기의 수를 컨트롤하는데 사용
- max_cat_group : 카테고리 수가 클 때, 과적합을 방지하는 분기 포인트를 찾음(디폴트 값 : 64)
- Task : 데이터에 대해서 수행하고자 하는 임무를 구체화, train일수도 있고 predict 예측일 수도 있음
- application : 문제 타입 설정, 디폴트는 회귀(regression: 회귀분석, binary: 이진 분류, multiclass: 다중 분류)
- learning_rate : 학습률(일반적인 값은 0.1, 0.001, 0.003 등)
- num_leaves : 전체 Tree의 leave 수 이고, 디폴트값은 31
- device : 디폴트 값은 CPU(GPU로 변경가능)



- 하이퍼파라미터를 설정하는 것은 모델링에서 매우 중요한 일
- 관계 있는 하이퍼파라미터들을 대상으로 가능한 모든 조합을 시도

주요 매개변수(Hyperparameter)

scikit-learn의 경우

GridSearchCV(모델, 모델의 파라미터목록, cv)

- cv : 교차검증 시 나눌 fold 수



Grid Search

- **best_params_** : Grid Search 후에 찾은 최고의 파라미터 값
- **best_score_** : 최고의 파라미터를 사용한 교차 검증 점수
- **best_estimator_** : 최고 성능의 모델

심화 학습

앙상블을 더 깊이 — OOB·조기 종료·스태킹·랜덤서치

심화 학습

앙상블을 더 깊이 — OOB·조기 종료·스태킹·랜덤서치

심화 학습

이미 배운 것

- 앙상블 = 여러 모델의 집단 지성
- 배깅(병렬) vs 부스팅(순차)
- 랜덤포레스트, XGBoost / LightGBM
- 그리드서치로 하이퍼파라미터 탐색

오늘 더 깊이

- ① 검증 데이터 없이 성능 재기 (OOB)
- ② 트리를 몇 개 만들어야 충분한가
- ③ 부스팅을 언제 멈춰야 하나 (조기 종료)

① OOB — 공짜로 얻는 검증 점수

별도 검증 세트 없이 성능을 추정한다

배깅은 데이터를 중복 허용으로 뽑는다 → 각 트리마다 '뽑히지 않은 데이터'가 약 37% 남는다. 이것을 그 트리의 시험지로 쓴다.

OOB (Out-of-Bag)

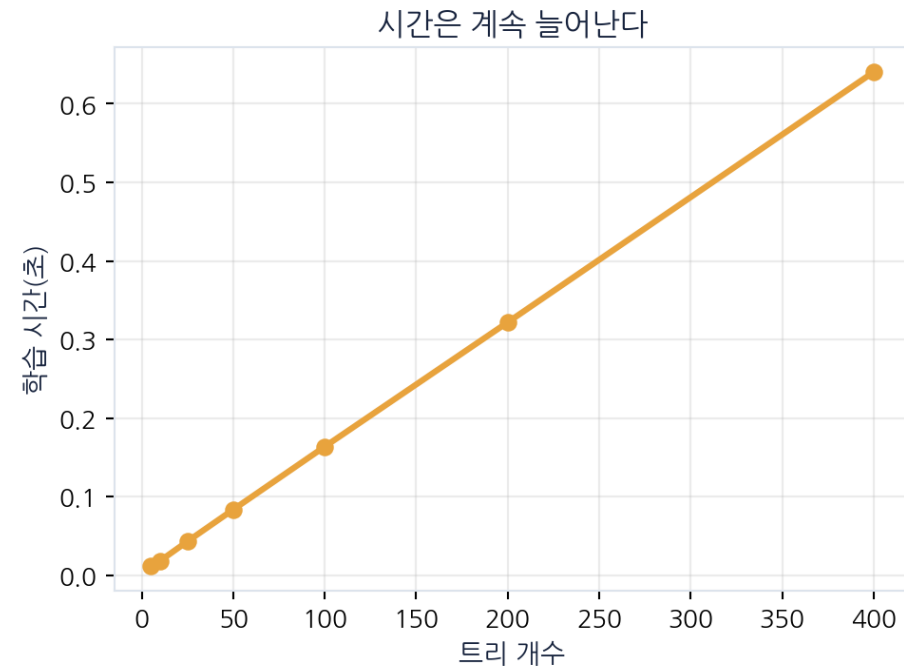
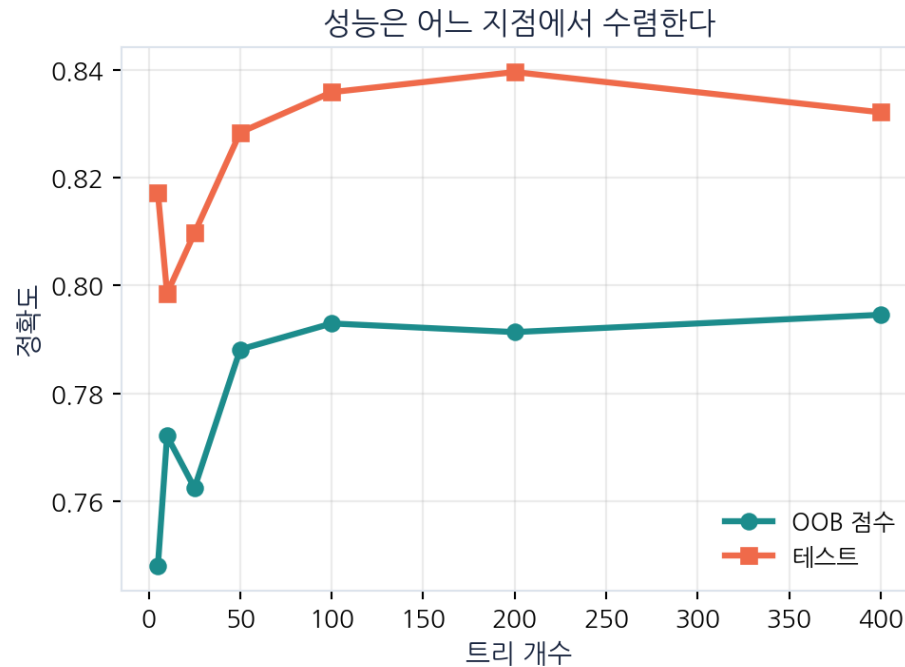
- 각 트리가 학습에 쓰지 않은 데이터로 채점
- 별도 검증 세트를 떼어낼 필요가 없다
- `oob_score=True` 한 줄이면 끝

언제 유용한가

- 데이터가 적어 검증셋 떼기 아까울 때
- 교차검증보다 훨씬 빠름 (한 번만 학습)
- 배깅 계열에서만 가능 (부스팅은 불가)

② 트리를 몇 그루 만들어야 충분한가

성능은 수렴하는데 시간은 계속 늘어난다



왼쪽: 성능은 100그루 근처에서 이미 평평해진다(수렴) · 오른쪽: 그런데 시간은 계속 선형으로 늘어난다

② 랜덤포레스트가 '랜덤'인 두 가지 이유

다양성이 앙상블의 생명이다

데이터 무작위

부트스트랩
트리마다 다른 샘플로 학습

특성 무작위

max_features
각 분할에서 일부 특성만 검토

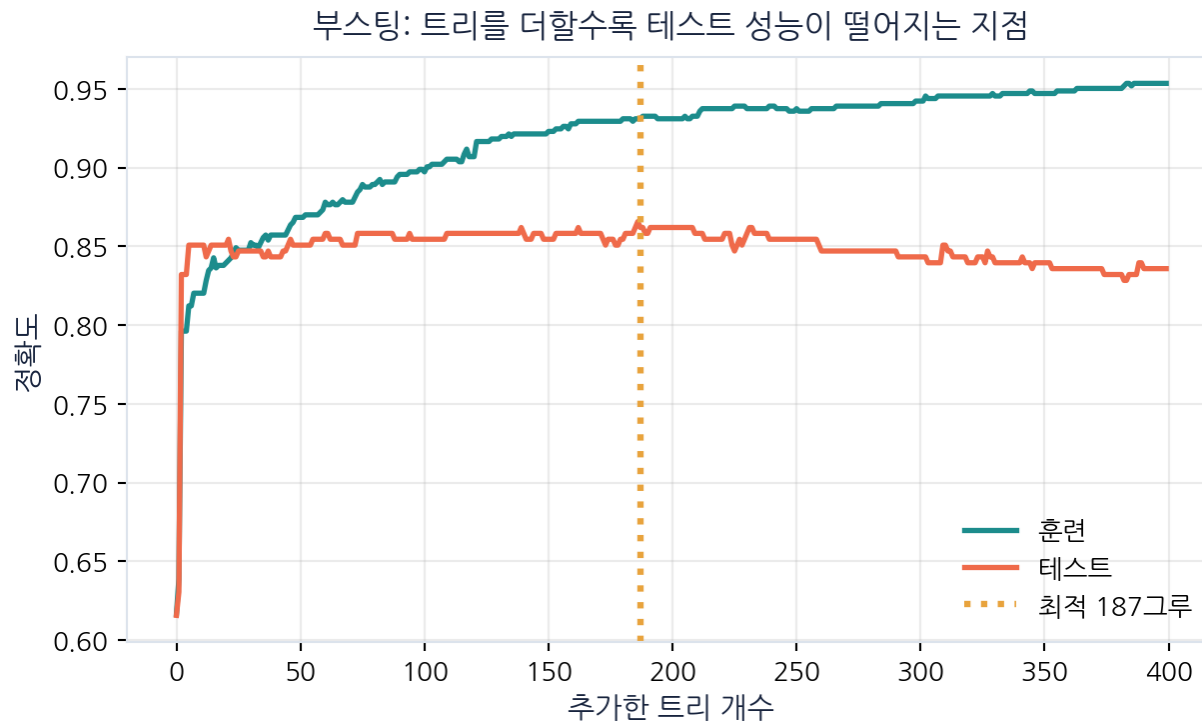
→ 다양성 확보

트리들이 서로 달라야
실수가 서로 상쇄된다

max_features가 작을수록 트리들이 더 달라진다 → 앙상블 효과 증가. None(전부 사용)이면 트리들이 비슷해져 효과가 준다.

③ 부스팅 — 트리를 더할수록 좋아지기만 할까?

어느 지점부터 테스트 성능이 떨어진다



staged_predict

- 부스팅은 순차적이라
 - '트리 n개까지 썼을 때'의 성능을
 - 전부 꺼내 볼 수 있다
 -
 - 훈련 성능은 계속 오르는데
 - 테스트는 어느 지점부터 하락
- 그 지점에서 멈춰야 한다

③ 두 손잡이는 서로 얽혀 있다

learning_rate와 n_estimators

learning_rate = 한 걸음의 크기

- 크면(0.5) : 빨리 학습, 거칠고 과대적합 위험
- 작으면(0.01) : 신중, 대신 트리가 많이 필요
- 실무 관행 : 0.05 ~ 0.1

조기 종료 (early stopping)

- 성능이 더 안 좋아지면 자동으로 멈춤
- n_iter_no_change 로 설정
- 시간 절약 + 과대적합 방지

실무 조합: learning_rate를 작게 + n_estimators를 넉넉히 + 조기 종료를 켜다

심화 정리 — 기억할 것

이제 노트북으로 직접 확인합니다

핵심 정리

- OOB = 검증셋 없이 얻는 공짜 점수
- 트리 개수는 수렴 지점까지만
- 부스팅은 과하면 오히려 나빠진다
- 조기 종료로 시간·과대적합 동시 해결
- 탐색은 랜덤 → 그리드 2단계

이제 실습으로

- 심화① OOB·트리 개수 수렴
- 심화② 부스팅 학습률·조기 종료

슬라이드에서 본 그래프가 여러분 화면에 그대로 나옵니다.